Attention Mechanisms

The breakthrough that changed NLP

Deep Learning Course

1 Introduction

2 The Attention Idea

3 Self-Attention: Query, Key, Value

4 Attention Visualization

5 Multi-Head Attention

6 Parallelization: The Key Advantage

7 Positional Encoding

8 The Transformer Block

Section 1

Introduction

The Problem: Sequential Bottleneck

RNNs process one word at a time, creating a bottleneck:

```
t=0: "The"     → h_1 (small state, remembers nothing)
t=1: "movie"   → h_2 (remembers "The", forgets distant past)
t=2: "was"     → h_3
...
t=100: "great" → h_100 (has to remember everything from 100 steps!)
```

Problems: - Can't parallelize (must process sequentially) - Distant context fades (vanishing gradients) - Slow training on long sequences

What if every word could "look at" every other word simultaneously?

Section 2

The Attention Idea

Soft Selection

Instead of reading left-to-right, use **attention** to focus on relevant words:

```
To understand "It was great", focus on:
- "great" → strong signal (positive sentiment)
- "was" → modifier (past tense context)
- "It" → subject (less important)
- Ignore: "a", "the" (filler words)
```

Attention as a spotlight: - Query: “What am I trying to understand?” (current position) - Keys: “What is each word?” (all words) - Values: “What information do they have?” (embeddings) - Output: Weighted average of relevant words

Section 3

Self-Attention: Query, Key, Value

The Math (Intuitive)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Step-by-step: 1. **Query** Q: "What position am I? What am I looking for?" 2. **Keys** K^T: "Match my query against all words" 3. **Softmax**: Convert to attention weights (0-1, sum=1) 4. **Values** V: Weighted combination of word vectors

Section 4

Attention Visualization

An Example

```
Query position: "was"

Sentence: "The movie was great"
Words:     The    movie   was   great

Step 1: Compute similarity (query vs. each key)
was vs The:    -0.2
was vs movie:   0.3
was vs was:     1.0  (match itself!)
was vs great:   0.5

Step 2: Apply softmax → [0.05, 0.15, 0.60, 0.20]
(Higher weights to "was" and "great")

Step 3: Weighted average of values
output = 0.05*v_the + 0.15*v_movie + 0.60*v_was + 0.20*v_great
```

Section 5

Multi-Head Attention

Multiple Perspectives

One attention head may not capture all relationships.

Solution: Use multiple heads, each learning different patterns

```
Head 1: Focus on semantic similarity ("movie" ↔ "film")
Head 2: Focus on grammatical role (subject, object, verb)
Head 3: Focus on sentiment modifiers ("very", "great")

# Each head is independent attention computation
# Concatenate outputs
attention_output = concat(head1, head2, head3, ..., head8)
```

Why multiple heads? - Capture different aspects of meaning - Robustness (if one fails, others compensate) - Parallel computation

Section 6

Parallelization: The Key Advantage

RNN vs. Attention

RNN (sequential):

```
Step 1: Process "The"    (1 GPU cycle)
Step 2: Process "movie"  (1 GPU cycle) ← must wait for step 1
Step 3: Process "was"    (1 GPU cycle) ← must wait for step 2
...
Total: 100 steps → 100 cycles
```

Attention (parallel):

```
Compute query, key, value for ALL words simultaneously
Compute attention scores for ALL pairs simultaneously
Result: Process whole sentence in a few cycles!
```

Speed gain: 10-50x faster on long sequences

Section 7

Positional Encoding

How Does Attention Know Order?

Attention ignores position (all words treated equally).

Solution: Add **positional encoding** to embeddings

$$PE(pos, 2i) = \sin(pos/10000^{2i/d})$$
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d})$$

Intuition: Each position gets a unique sinusoidal pattern

```
Position 0: [sin(0), cos(0), sin(0), cos(0), ...]
Position 1: [sin(1), cos(1), sin(1), cos(1), ...]
Position 2: [sin(2), cos(2), sin(2), cos(2), ...]
...
```

Why sinusoids? Periodicities at different scales capture relative positions

Section 8

The Transformer Block

Putting It Together

```
Input → Positional Encoding → Multi-Head Attention → Add & Norm
   ↓
Feed-Forward Network → Add & Norm → Output
```

Add & Norm: Residual connection + layer normalization

$$\text{Output} = \text{LayerNorm}(x + \text{Attention}(x))$$

Why residual? - Gradients flow better (addition vs. composition) - Network learns "refinements" not absolute transformations

Section 9

Attention in Code

Implementing Self-Attention (Simplified)

```python
import tensorflow as tf
from tensorflow import keras

class SelfAttention(keras.layers.Layer):
    def __init__(self, d_model):
        super().__init__()
        self.d_model = d_model
        self.query = keras.layers.Dense(d_model)
        self.key = keras.layers.Dense(d_model)
        self.value = keras.layers.Dense(d_model)

    def call(self, x):
        # x shape: (batch, seq_length, d_model)
        Q = self.query(x)
        K = self.key(x)
        V = self.value(x)
```

Section 10

Transformer Layer in Keras

Using Built-in Layers

```
# MultiHeadAttention is built-in (TensorFlow 2.x)
attention_layer = keras.layers.MultiHeadAttention(
    num_heads=8,
    key_dim=64  # d_model / num_heads
)

# Self-attention on sequence
outputs = attention_layer(queries=x, values=x, keys=x)
```

Parameters: - `num_heads`: how many independent heads - `key_dim`: dimension per head (total = key_dim × num_heads)

Section 11

Practical Example: Sentiment with Attention

Task and Setup

Goal: Sentiment classification with explicit attention mechanism

Architecture: 1. Embedding layer 2. Positional encoding 3. Self-attention block 4. Feed-forward 5. Classification head

Section 12

Practical Example: Full Model

Implementation

```
import tensorflow as tf
from tensorflow import keras

vocab_size = 5000
embedding_dim = 128
seq_length = 200
num_heads = 8

# Build attention-based classifier
model = keras.Sequential([
    keras.layers.Embedding(vocab_size, embedding_dim),

    # Add positional encoding (simplified: just concatenate position)
    # Proper positional encoding: see next slide

    keras.layers.MultiHeadAttention(
```

Section 13

Positional Encoding Implementation

Adding Position Information

```
def positional_encoding(position, d_model):
    import numpy as np
    angle_rads = positional_encoding_angles(
        np.arange(position)[:, np.newaxis],
        np.arange(d_model)[np.newaxis, :],
        d_model
    )
    # Apply sin to even indices
    angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
    # Apply cos to odd indices
    angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])
    return angle_rads[np.newaxis, ...]

def positional_encoding_angles(pos, i, d_model):
    import numpy as np
    return pos / np.power(10000, (2 * (i//2)) / np.float32(d_model))
```

Section 14

Training and Evaluation

Complete Training Loop

```
# Load IMDb data
(x_train, y_train), (x_test, y_test) = \
    keras.datasets.imdb.load_data(num_words=vocab_size)

# Pad sequences
x_train_padded = keras.preprocessing.sequence.pad_sequences(
    x_train, maxlen=seq_length
)
x_test_padded = keras.preprocessing.sequence.pad_sequences(
    x_test, maxlen=seq_length
)

# Train
history = model.fit(
    x_train_padded, y_train,
    epochs=3,
```

Section 15

Attention Visualization

Seeing What the Model Attends To

```python
# Extract attention weights from layer
# Create a model that outputs attention weights
attention_layer = model.layers[1]  # MultiHeadAttention

def get_attention_weights(x):
    # Build custom model to access attention
    inputs = keras.Input(shape=(seq_length,), dtype='int32')
    x = keras.layers.Embedding(vocab_size, embedding_dim)(inputs)
    attention_out, attention_weights = attention_layer(
        x, return_attention_scores=True
    )
    attention_model = keras.Model(inputs, attention_weights)
    return attention_model.predict(x)

# Visualize
weights = get_attention_weights(x_test_padded[:1])
```

Section 16

Attention vs. Earlier Approaches

Performance and Speed Comparison

Model	Accuracy	Time/Epoch	Parallelizable
Averaged Embed	0.88	2 sec	✓ (all at once)
LSTM	0.92	45 sec	□ (sequential)
CNN	0.91	8 sec	✓ (parallel)
Attention	0.92	10 sec	✓ (fully parallel)

Attention advantages: - Accuracy comparable to LSTM - Much faster (parallelizable) - Better for long sequences - More interpretable (attention weights)

Section 17

Key Concepts Summary

Attention Recap

- Soft selection: focus on relevant words
- Query-Key-Value framework
- Multi-head attention: multiple perspectives
- Positional encoding: preserve order
- Self-attention in transformers
- Parallelizable architecture

Why This Matters

Attention is the foundation of modern NLP (BERT, GPT, etc.). Understanding it is critical for Class 5.

Section 18

Common Issues

Troubleshooting Attention

Problem	Cause	Solution
All attention to one word	Under-training	Train longer, higher LR
Uniform attention (no focus)	Model too simple	Add more heads, deeper network
Memory error	Too many heads × dimensions	Reduce key_dim or num_heads
Slow training	Implementation inefficiency	Use Keras built-ins, not custom

Section 19

Exercises for Class 4

Hands-On Tasks

1. **Visualize attention**: Generate attention weights for 10 examples. Do patterns make sense?
2. **Vary num_heads**: Try 2, 4, 8, 16 heads. Does more always help?
3. **Compare to LSTM**: Train LSTM with same parameters. Speed vs. accuracy?
4. **Positional encoding ablation**: Train without positional encoding. How much does it help?
5. **Attention interpretability**: For misclassified examples, what did the model attend to?
6. **Scale test**: Train on sequences of length 100, 500, 1000. Attention vs. LSTM speed?

Section 20

Next Class Preview

From Building Blocks to Pre-trained Models

Attention is powerful, but training from scratch is expensive:

```
Training BERT from scratch:
- 100+ GB of text
- 4-16 TPUs/GPUs
- Weeks of training
- Billions of dollars
```

Class 5: Transformers & Pre-trained Models - Full transformer architecture (encoder + decoder) - Pre-trained models (BERT, GPT, etc.) - Transfer learning (fine-tuning) - Using HuggingFace

Section 21

Questions & Discussion

Questions & Discussion

Thank you!

Key takeaway: Attention mechanisms let every word see every other word instantly. This parallelization + expressiveness is why transformers dominate modern NLP.

Next: Complete transformer models and pre-training